

Final Project Report: Stream Analysis

Federico Maspero

Department of Computer Science, University of Milan

Dario Malchiodi

June 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content

1. Introduction

This report describes the results obtained in the final project for the Algorithms for Massive Datasets exam. The project, chosen from a list provided by Professor Malchiodi, was focused on implementing two or more algorithms for stream analysis described during the course.

Stream analysis is a technique utilized to process large amounts of data that would be too large to store. Each of the presented algorithms allows to analyze the data by reading it in a single-pass stream and utilizing a much more limited amount of memory.

The implemented algorithms, all described in *Mining of Massive Datasets* [1], were applied to a dataset of articles and their user comments (detailed in Section 2). They are: the Bloom filter, used to evaluate whether an article has been seen before; the Flajolet-Martin algorithm (FM), used to estimate the total number of users who have left comments on the articles; and the Alon-Matias-Szegedy algorithm (AMS), used to calculate the second moment of the article sections.

The results were evaluated against ground truths calculated from the same subsample of data.

This report aims to show the methodology and algorithms used in the project

[2], evaluate the experiment results, and prove the memory-usage efficiency of these methods.

2. Dataset and data organization

This section will introduce the dataset selected for the project, as well as illustrate how the data treated, organized and pre-processed

2.1 The dataset

The dataset utilized for the project is the *New York Times Articles & Comments (2020)*, published on Kaggle [3] under the CC BY-NC-SA 4.0 license [4]. The dataset was first accessed for the project on June 13, 2026.

The dataset is not stored on the GitHub repo of the project, but is dynamically downloaded at runtime via the Kaggle API. Once downloaded, the project will detect the presence of the file and not re-attempt the download on subsequent runs.

Of the entire dataset, the comments file `nyt-comments-2020.csv` was utilized as the main data stream, while the article file `nyt-articles-2020.csv` was used for the section lookup. The dataset also contains pre-split partitions of the comments file and additional train/test files, which were not used.

2.2 Data organization and pre-processing

As previously mentioned, the project is based on stream analysis, a data analysis model in which the input data is read in a single pass and processed immediately, allowing to not load the full dataset into memory.

This method is applied to the comments file, `nyt-comments-2020.csv`. A generator was built to yield one comment at a time by lazy-reading the file. The single comment row is composed of 23 columns, shown in the Jupyter notebook. Of these columns, the algorithms particularly use the `userID` (FM) and `articleID` (Bloom filter) attributes.

AMS makes use of the `section` attribute found in the `nyt-articles-2020.csv`. To correlate each comment to its section, a dictionary was built to map the two attributes `articleID` and `section`. When this mapping is not possible (articles from outside the dataset's coverage, missing values in the articles file) the `articleID` is mapped to `UNKNOWN`, so as not to drop any articles from the stream.

A `SUBSAMPLE` flag was created to allow the user to choose whether to use the full file or a sampled portion. When this flag is set to `True` (as by default), the `limit` variable will be set to 500000. It is possible to change the flag to

`False` to test the algorithms on the full dataset.

Some experiments, however, use a different subsampling, set by the variable `EXP_LIMIT = 50000`. This was done to lower computational cost: these particular experiments perform multiple runs over the given stream of data to compare results at different hyperparameters, so a smaller stream was necessary to keep the computation times acceptable.

3. Algorithms and Implementation

This section will highlight the algorithms used in the project, the theory they are based on and the way they were implemented.

All randomized parts of the implementation use a fixed random seed, set once at initialization, so that results are reproducible across executions.

3.1 Bloom Filter

The Bloom filter is a probabilistic data structure used to check set membership in a memory-efficient way.

When given an item, a Bloom filter is able to tell if said item has "definitely never been seen before", or "possibly seen before", guaranteeing no false negatives at the cost of some false-positives.

To do this, a Bloom filter uses a bit-array and a set number of hash functions. Instead of storing full seen elements, it creates an imprint by hashing them, and stores the result. That is why this system must accept to have some false-positives: some elements may share the same imprint because of hash collision. Meanwhile, false-negatives are impossible because when an item is genuinely inserted, `add` sets all k of its bit positions to 1, making sure an inserted item can never return `False`.

The specific way a bloom filter works is:

```
Add(x):
    for i in 0..k-1:
        bits[ h_i(x) ] = 1

Check(x):
    for i in 0..k-1:
        if bits[ h_i(x) ] == 0:
            return False      # certain: never inserted
    return True               # possibly inserted (may be
                             # false-positive)
```

This false-positive rate can be predicted and bounded by choosing adequate hyperparameters. The false-positive probability after inserting n elements in

bit-array of size m is approximately:

$$P(FP) \approx (1 - e^{(-kn/m)})^k$$

The project's implementation of the bloom filter closely follows this structure, with the functions `add` and `check` specifically implementing the pseudo-code previously presented. The main design decision lies in the `_hashes` function, in particular in how the k hash functions are obtained, each coming from one base hash, salted with the function index:

```
def _hashes(self, item):
    values = []
    for i in range(self.num_hashes):
        combine = f"{i}_{item}"
        digest =
        hashlib.md5(combine.encode()).hexdigest()
        h = int(digest, 16)
        position = h % self.size
        values.append(position)
    return values
```

3.2 Flajolet-Martin Algorithm

The Flajolet-Martin Algorithm (FM) is used to estimate the number of distinct elements in a stream using space far smaller than the count itself. This is done by taking advantage of some characteristics of the randomness of hash functions, in particular:

- Determinism: the same element always produces the same hash
- Uniform distribution: hash values are uniformly distributed across the binary space

The intuition: if elements are hashed to uniformly random bit-strings, the rarer a pattern is, the more distinct elements must have been seen for it to appear. Specifically, a hash value ending in r zeros occurs with probability $2^{(-r)}$, so observing a maximum trailing-zero run of R across the stream suggests roughly 2^R distinct elements.

```
Add(x):
    for i in 0..k-1:
        r = trailing_zeros( h_i(x) )
        max_tails[i] = max(max_tails[i], r)

Estimate():
    group the 2^max_tails values,
```

```
take the median of each group,  
average the medians
```

The accuracy of the final estimation comes from combining many independent estimators, by grouping the hash results and performing an average of the median of each group. This process leads to removing outliers resulting from the hashing process and reaching a more faithful estimation.

The project's implementation follows this described flow of actions. The tail length is calculated with the bitwise expression `(x & -x).bit_length() - 1`, isolating the lowest set bit. The k hash functions are obtained in the same way as per the bloom filter, using `hashlib.md5` to ensure independence between them. Both the number of hashes and group size are left as tunable hyperparameters

```
def add(self, item):  
    for i in range(self.num_hashes):  
        combine = f"{i}_{item}"  
        digest =  
hashlib.md5(combine.encode()).hexdigest()  
        h = int(digest, 16)  
        tail_length = self._tail_length(h)  
        if tail_length > self.max_tails[i]:  
            self.max_tails[i] = tail_length  
  
def estimate(self, group_size):  
    estimates = []  
    for r in self.max_tails:  
        estimates.append(2**r)  
    group_medians = []  
    for start in range(0, len(estimates), group_size):  
        group = estimates[start : start + group_size]  
        m = median(group)  
        group_medians.append(m)  
    return sum(group_medians)/len(group_medians)
```

3.3 Alon-Matias-Szegedy Algorithm

Moments are statistical metrics used to capture the shape of a distribution, based on the frequencies of distinct items in a stream.

The second moment ($k = 2$): $\sum_i (m_i)^2$ allows to measure how uneven or skewed a data distribution is. The resulting value is called surprise number.

The Alon-Matias-Szegedy Algorithm (AMS) algorithm estimates the second moment by randomly sampling just a few specific positions in the stream. In this project's case, the second moment is calculated over the distribution of

comments among articles belonging to different sections, those sections being considered as classes.

```
Add(x):
    t=t+1
    for i in 0...k-1
        with probability 1/t: replace stored element with
x, set count to 1
        else if x == stored element: add 1 to count

Estimate():
    for i in 0...k-1
        compute t*(2*element count -1)
    return average of results
```

The expected value (the statistical average) of every random variable X is exactly the true second moment F_2 . By tracking multiple independent variables and averaging their estimates, we can get an accurate estimate of F_2 with high probability: the estimator is $t(2v - 1)$. When calculating out the expected value over all positions, for an element appearing m times the contributions sum as $1 + 3 + 5 + \dots + (2m - 1)$, which equals m^2 (sum of first m odd numbers). Sum over all elements $\rightarrow \sum m^2 = F_2$.

To handle an infinitely growing stream of variables to account for, AMS must use a dynamic sampling method. That is reservoir sampling, a technique that maintains a reservoir of k elements that are currently being tracked (the sample) and guarantees an invariant: at any exact moment in time, every element seen so far has the same probability of being in the reservoir, having $p = 1/t$, where t is the number of seen elements

In the project, these concepts were implemented in the following ways: the number of variables is kept track via two parallel lists, having `vars_element` track the elements themselves and `vars_count` holding their counts. The reservoir sampling was implemented through a random function `random.random() < 1/t`, seeded as to ensure reproducibility of the results across runs.

```
def add(self, element):
    self.t += 1
    for j in range(self.num_vars):
        if random.random() < 1/self.t:
            self.vars_element[j] = element
            self.vars_count[j] = 1
        elif element == self.vars_element[j]:
            self.vars_count[j] += 1

def estimate(self):
```

```

res = []
for j in range(self.num_vars):
    est = self.t * (2 * self.vars_count[j] - 1)
    res.append(est)
return sum(res)/len(res)

```

4. Experiments

This section will describe the experiments performed on the `nyt-comments-2020.csv` dataset. These experiments are two hyperparameter sweeps for FM and AMS, a full run of Bloom filter on a set of `article_id` to measure false positive rates at different hyperparameters, and a run of FM and AMS on the full subsampled stream of data with set hyperparameters, to measure accuracy results compared to the ground truth.

4.1 FM hyperparameter sweep

In this experiment, the number of true distinct users was measured on the sample of dataset bound by `EXP_LIMIT` (as mentioned in section 2). This value, 20108 users, was then used to calculate the error rate of the estimation performed by the implemented FM algorithm at various `num_hashes` settings, in particular `[64, 128, 256, 512, 1024, 2048]`. The results, showcased by the graph in figure 1, demonstrate how the estimation gets better the more hash functions are utilized, reaching a 0.69% error rate at 2048 hashes.

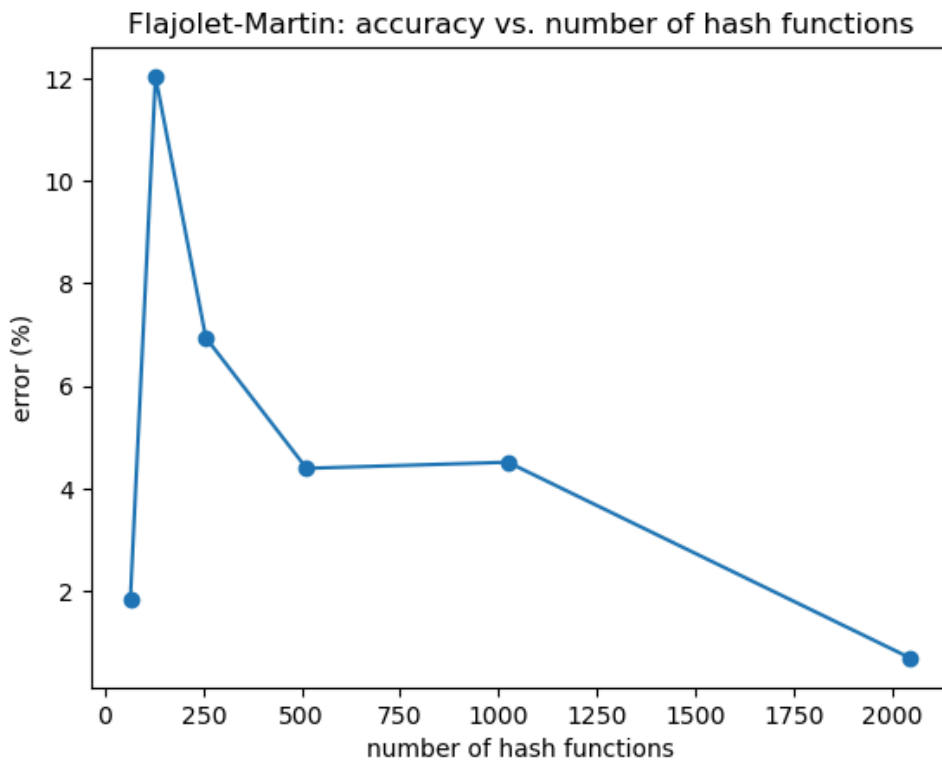


Figure 1: Graph of FM results accuracy over number of hash functions

Note how the algorithm shows an extremely good approximation at just 64 hashes, before rising sharply at 128 and slowly decreasing, following the expected trend. This is explained by the variance of the FM estimator: each result is a single random draw tied to the specific hash family used, and that variance is high at low `num_hashes` and shrinks as more are added. The downward trend is real (more hashes = lower variance = better estimate), but individual points scatter around it, and the lucky 64 reflects a favorable hash draw rather than a real ranking between settings.

4.2 AMS hyperparameter sweep

This experiment follows closely the structure of the previous one. It first calculates the second moment on the `EXP_LIMIT` subsection of the dataset, obtaining a ground truth ($F_2 = 565787146$). It then calculates the error rate of the implemented AMS algorithm at various settings of `num_vars`, specifically `[10, 25, 50, 100, 200, 500, 1000]`. In this case, the results are calculated by averaging the results of 10 runs over each hyperparameter. That is to reduce the noise introduced by the randomness of the reservoir sampling of AMS. The results of the various runs are showcased by the graph in figure 2.

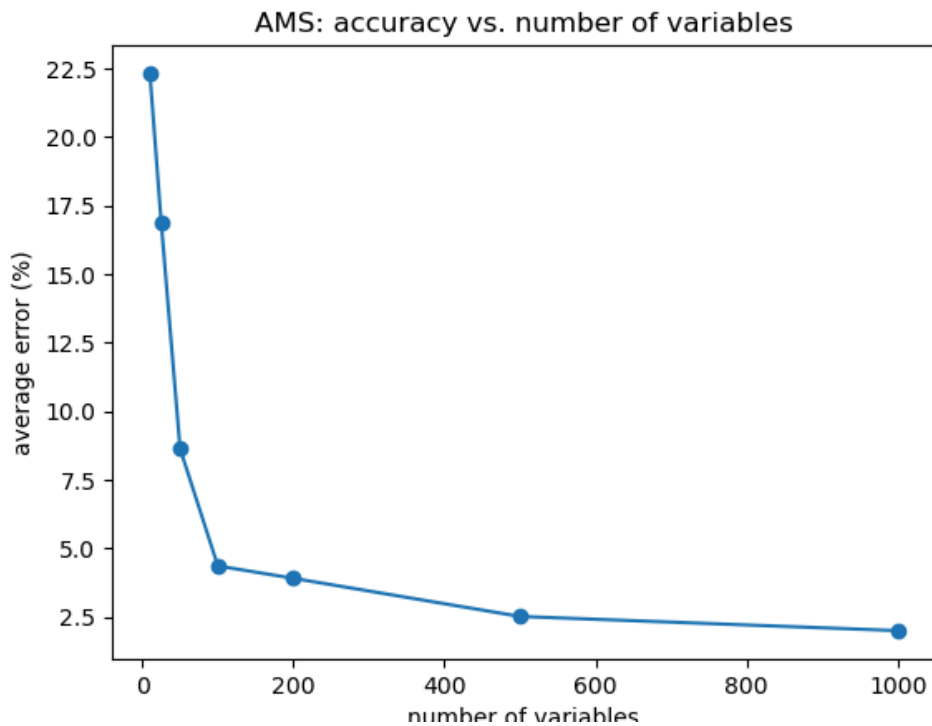


Figure 2: Graph of AMS results accuracy over number of variables

The graph shows how the error significantly decreases as `num_vars` grows, but it also highlights diminishing returns: there is a steep improvement early on, between the 10 to 200 variables runs, which flattens later on in the last three runs (200 to 1000 variables), only showing a 2% improvement.

It is important to note the biggest difference between these two experiments: by averaging the results across multiple runs, the final curve described by the

AMS results is far smoother than the jagged one of the deterministic results of FM. The average in this case is absolutely necessary, due to the intrinsic randomness of AMS, and allows the final curve to more faithfully represent the algorithm's true accuracy at each setting by cancelling out per-run noise.

4.3 Bloom filter accuracy

This experiment differs structurally from the previous ones, given that Bloom filter doesn't estimate a quantity to verify against a ground truth, but it verifies set membership. What this experiment was set up to do is calculate the empirical false-positive rate obtained by each iteration, and test it against the theoretical formula $(1 - e^{(-kn/m)})^k$ described in *Mining of Massive Datasets*.

To calculate these results, a pass of the full `nyt-comments-2020.csv` was performed to collect all distinct `articleID`, then split them into two disjoint halves: one set added to the filter (`members`), one set never added (`non_members`).

In this case, the full file was scanned instead of a subsampled portion because neither `EXP_LIMIT` or `limit` limiters contained enough distinct articles to compute meaningful results. The final count was divided into 8393 `members` and 8394 `non_members`.

The algorithm was then run over different `size` values: [2000, 5000, 10000, 30000, 100000, 300000], with a fixed `num_hashes` of 3. The results of the empirical and theoretical false-positive rates are shown by the graph of figure 3.

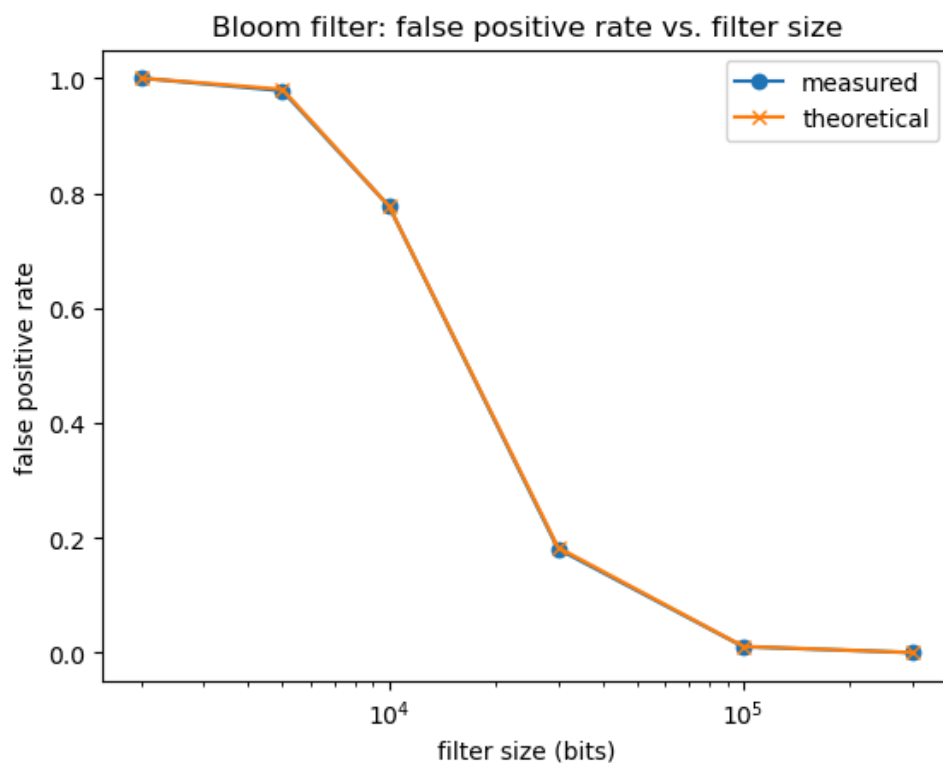


Figure 3: Graph of the empirical and theoretical false-positive rates for Bloom filter

The image shows that the empirical and theoretical curves overlap almost perfectly across the entire range. It is also possible to see how at small sizes the false-positive rate is near 1.0, dropping toward ~ 0 as size grows. Overall, this shows that the implementation of the algorithm yielded extremely close results to the ones expected by the mathematical theory behind it.

4.4 Full stream run of FM and AMS

This final experiment was set up to demonstrate the behavior of the algorithms when run over a large stream of data. In this case, FM and AMS are set to read over the full 500000 element stream, while at the same time the ground truths for the whole stream are calculated (the exact distinct-user count and second moment). At the end, these results are compared, and the margin of error of the two final results is computed.

The two algorithms are set up with fixed hyperparameters, in particular `num_hashes = 256` for FM and `num_vars = 200` for AMS. These parameters were chosen because they showed acceptable performances in the previous hyperparameter sweep experiments (sections 4.1 and 4.2) while keeping computation costs relatively low - higher hyperparameters were considered and tried, but led to far too long computation times (over 35-40 minutes) and were deemed not acceptable for this experiment.

The final results showed an error of 5.9% for FM and a 1.7% for AMS. Both algorithms managed to estimate their target within single-digit error on a 500000 element stream in a single pass, using bounded memory. This demonstrates that the approximate methods remained accurate at scale. Section 6 will compare the results of the two algorithms with the ones calculated in the hyperparameter sweep.

5. Scalability

This section will discuss the scalability of the three implemented algorithms, why they scale so efficiently even with larger data size, and will showcase proof of this characteristic.

As briefly explained in section 2, all these algorithms are based on a data streaming design, specifically thought to analyze large amounts of data. In this method, data is read one element at a time through a generator, so the input is never held in memory regardless of file size. The streaming algorithms use memory that is bounded and constant in the stream length, while exact computation requires memory that grows linearly with the number of distinct elements, so only the streaming approach scales to

arbitrarily large or unbounded data. This is visible in the graph represented in figure 4, which shows the memory usages through the experiment

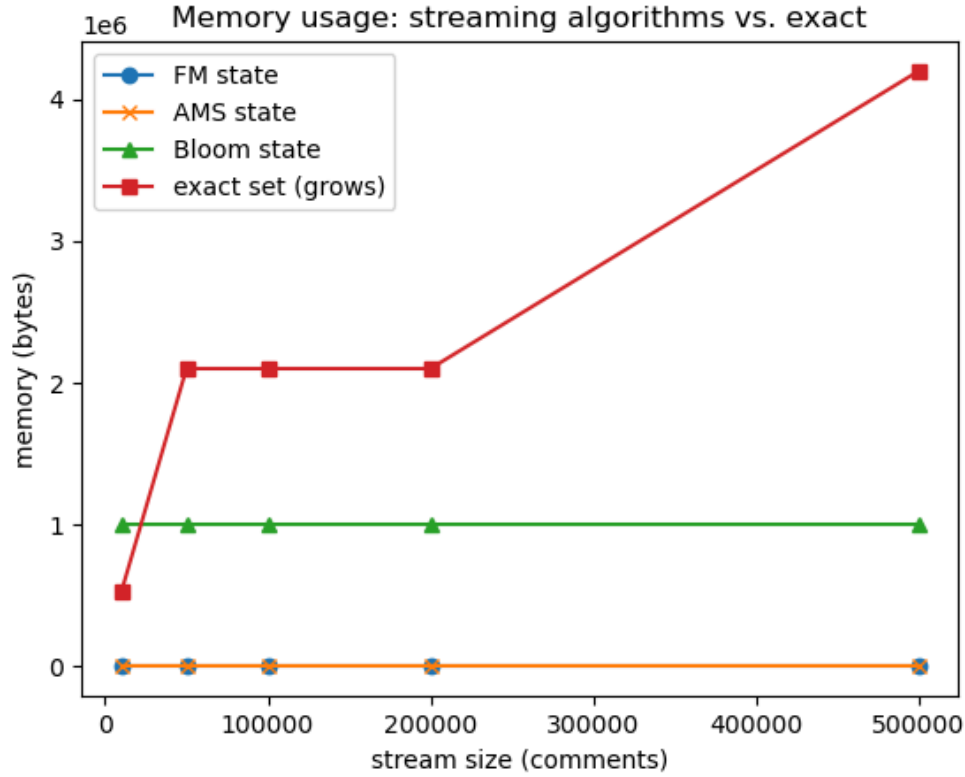


Figure 4: Graph of memory usage

As shown by the graph, the three algorithms stay flat (FM ~2KB, AMS ~3KB, Bloom ~1MB) across all stream sizes, while the exact set grows (to 4MB at 500k, subsample set for the experiment, but the trend shows that it would continue to grow without bound on larger data).

FM keeps `num_hashes` integers, AMS keeps `num_vars` element/count pairs: none grow as more data flows, which allows their memory usage to be kept extremely low. Notice how the Bloom filter uses more memory than the other two, since it does require the bit-array `bits` of fixed size to be created. This creates a situation where the exact set uses less data than the Bloom filter. However, as it is clearly visible, the exact set is smaller only at first, but its unbounded growth means the streaming algorithms win as data scales, since the bound on the size of `bits` will never grow.

One thing to note is that the memory usage measurements are obtained through `sys.getsizeof`, which is an approximate measuring tool (it sizes the container, and the set grows in steps due to Python's internal resizing). This explains the three-points plateau where the exact set doesn't grow (the exact values probably were between ~0.5 and ~2.1 MB).

6. Comments and discussion of results

This section will discuss some insights observed during the preparation of the experiments and after the study of the results.

The results of the experiment described in section 4.1 were overall correct, but show a real limitation of the algorithm: FM is deterministic, so unlike AMS its results cannot be smoothed by averaging without introducing additional randomness beyond the course's method; the scatter is therefore an inherent property of the deterministic estimator.

Something noticed in section 4.4 was that the main-run results are consistent with the sweep results within normal variation, especially given AMS's randomness and FM's scatter, confirming the algorithms scale to the larger stream without loss of accuracy.

In general, all experiments show that higher hyperparameters improve accuracy, but at full-stream scale they become much more computationally expensive. All this highlights the importance of tuning hyperparameters to find a satisfying compromise between results accuracy and computational cost.

The final results are overall quite positive: FM and AMS both reach single-digit errors in both settings they were tested in, while Bloom filter matches the theory. All of these demonstrate that the streaming method works, and the scalability experiment shows that the method succeeds in its purpose of maintaining low memory usage over large amounts of data.

7. Conclusion

In conclusion, this project manages to showcase the streaming method and its efficiency in memory management over large amounts of data.

To do that, three streaming algorithms were implemented from scratch: Bloom filter, Flajolet-Martin algorithm, and Alon-Matias-Szegedy algorithm. They were tested on the *New York Times Articles & Comments (2020)* dataset, treated as a stream.

The results were satisfying for all experiments, with Bloom's false-positive rate matching theory, and FM and AMS estimating their targets within single-digit error of the exact ground truth.

The scaling analysis confirmed bounded memory regardless of stream size, showing these methods achieve good accuracy at a fixed memory cost that doesn't grow with data, proving that streaming algorithms trade a small amount of accuracy for the ability to process large (or unbounded) data within a fixed and small memory budget.

Bibliography

- [1] *Mining of Massive Datasets 2nd edition* - J. Leskovec, A. Rajaraman, J. Ullman <http://www.mmds.org/>
- [2] Project's Github repository
https://github.com/Femaspe02/AlgorithmsProject_MasperoFederico
- [3] "New York Times Articles & Comments" Kaggle repository
<https://www.kaggle.com/datasets/benjaminawd/new-york-times-articles-comments-2020>
- [4] CC BY-NC-SA 4.0 license <https://creativecommons.org/licenses/by-nc-sa/4.0/>